

Generative AI and Applications

Handson-03

Name: Chethan S	SRN: PES2UG23CS150
Section: 6 c	Date: 02-04-2026

1. Advanced RAG and Retrieval:

Here, we focused on improving the retrieval stage of RAG, not generation. It shows why naïve dense-only retrieval fails on exact keyword queries and why BM25 alone fails on semantic queries, then builds up to SBERT and ColBERT as dense retrievers and RRF-based hybrid retrieval as the main solution.

The notebook walks through a full progression: dense retrieval demonstration, BM25 theory and scoring, ColBERT MaxSim late interaction, and then a reusable HybridRetriever that fuses BM25 and SBERT rankings. It includes worked numerical examples so the rank fusion is explained step by step instead of just presented abstractly.

Key Takeaways:

- ❖ Sparse retrieval (BM25) = bag-of-words, term statistics, exact match power.
- ❖ Dense retrieval (SBERT/ColBERT) = neural embeddings, semantic understanding.
- ❖ Hybrid retrieval (BM25 + SBERT + RRF) = combines both by fusing rank lists, not raw scores.
- ❖ RRF formula: $\text{score}(d) = \sum 1/(k + \text{rank}(d))$ — simple, powerful, no hyperparameter tuning needed.

2. Reranking and Query Expansion:

We start by showing the role of cross-encoder re-ranking. The notebook contrasts bi-encoders like SBERT, which encode query and document separately for speed, with cross-encoders, which score the full query-document pair for better relevance. It then demonstrates a two-stage retrieval flow: fast first-stage retrieval with SBERT, followed by cross-encoder re-ranking to keep only the most relevant passages.

The second major topic is query expansion. It covers two strategies: HyDE, where Gemini generates a hypothetical answer that is embedded and used for retrieval, and Multi-Query **retrieval**, where Gemini generates several paraphrases of the same question and the retrieved results are merged. Both are meant to help with vague or underspecified user queries.

The notebook finishes by wiring everything together into a production-style pipeline:

Query expansion → hybrid retrieval → re-ranking → LLM generation.

Key takeaways:

Retrieval quality improves substantially when you first broaden the query, then fuse sparse and dense retrieval, and finally re-score the best candidates with a stronger model before generating the final answer

Comparison of Approaches

Stage	Naïve RAG	Advanced RAG
Query	Raw user input	HyDE expanded / multi-phrasing
Retrieval	Dense only (cosine)	Hybrid (BM25 + SBERT + RRF)
Candidate filtering	Top-K by embedding score	Cross-encoder re-ranking
Generation	All top-K in context	Only highest-quality docs

3. Generation Enhancement and MOE notebook:

We covers four main techniques:

- Fine-Tuning (LoRA/PEFT): Efficiently adapts LLMs to new tasks or domains by training small adapter matrices instead of all weights.
- Precision and Quantization: Reduces model size and speeds up inference by lowering numerical precision (FP32 → FP16/BF16/INT8/INT4), with practical demos and memory/accuracy trade-offs.
- Mixture of Experts (MoE): Uses a router to select specialized “expert” models or prompts for different query types, increasing both efficiency and quality.
- Software MoE with LangChain: Implements a router and multiple expert chains (math, code, creative, science, general) to dispatch queries to the best-suited LLM configuration.

Key takeaways:

Generation quality, efficiency, and specialization can be dramatically improved by combining fine-tuning, quantization, and expert routing, completing the Advanced RAG pipeline.

Generation Enhancement — Full Picture

Technique	What It Does	When to Use	Key Library
Full Fine-Tuning	Updates all weights	Large budget, very specialized domain	<code>transformers</code> , <code>Trainer</code>
LoRA / PEFT	Trains tiny adapter matrices (< 1% of params)	Medium budget, task adaptation	<code>peft</code>
FP16 / BF16	Half-precision training/inference	Always — halves memory, negligible quality loss	<code>torch.float16</code>
INT8 Quantization	8-bit weights (4x memory reduction)	Inference on consumer hardware	<code>bitsandbytes</code>
INT4 / GPTQ	4-bit weights (8x memory reduction)	Running 70B models on a single GPU	<code>bitsandbytes</code> , <code>auto-gptq</code>
Software MoE	Routes queries to specialist prompts	Multi-domain chatbots, enterprise assistants	LangChain
Model MoE	Sparse expert activation per token	Best quality-per-FLOP at scale	Mixtral, GPT-4